

12. ETAPA 8 — Desarrollo del software

En esta etapa nos enfocamos en desarrollar el sistema completo y cumplir con los requisitos mínimos que se pedían. Trabajamos con una arquitectura de Frontend + Backend + Base de Datos. El Frontend está desarrollado con Angular y se encarga de la parte visual e interacción con el usuario, el Backend utiliza Node.js, Express y TypeScript para manejar la lógica, las validaciones y la seguridad, y PostgreSQL se encarga de almacenar la información. A continuación se explica cómo se implementó cada uno de los mínimos.

Autenticación

Para el inicio de sesión implementamos JWT (JSON Web Token). Cuando el usuario ingresa su correo y contraseña, el Backend busca sus datos en la base de datos y compara la contraseña utilizando bcryptjs. Las contraseñas no se guardan directamente, sino como un hash. Si los datos son correctos, el Backend genera un token firmado con la clave secreta JWT_SECRET, donde se incluye información como el id, correo y rol del usuario.

El Frontend guarda este token y lo utiliza en las peticiones que necesitan autenticación. Para esto utilizamos un interceptor de Angular que agrega el token en el header Authorization. En el Backend también tenemos el middleware auth.middleware.ts, que revisa si el token existe y si sigue siendo válido. Si no se envía, es incorrecto o ya expiró, se responde con un error 401 y la petición no continúa. Si el token es válido, se obtiene la información del usuario y se permite continuar.

También agregamos un middleware de roles llamado roleMiddleware, que se utiliza en las rutas donde se necesita restringir el acceso. Por ejemplo, la creación de usuarios está limitada al rol de Administrador.

CRUD

En los módulos principales implementamos las operaciones CRUD: crear, listar, consultar, editar y eliminar. Esto se aplica a Incidentes, Usuarios, Refugios, Recursos, Zonas, Alertas, Historial, Asignación de Personal y Asignación de Recursos.

En el Frontend cada módulo tiene sus componentes correspondientes. Por ejemplo, en Incidentes tenemos incidente-form, incidente-list y incidente-detail. También agregamos incidente-map para visualizar los incidentes en un mapa y incidente-stats para mostrar algunas estadísticas.

En el Backend cada módulo cuenta con sus rutas y controladores. Estos reciben las peticiones, utilizan el servicio correspondiente y realizan las operaciones necesarias en la

base de datos. Después se devuelve una respuesta al Frontend indicando si la operación se realizó correctamente o si ocurrió algún problema.

Validación de datos

Las validaciones se realizaron tanto en el Frontend como en el Backend. En Angular, los formularios revisan que los campos obligatorios estén completos y que los datos tengan el formato correcto. Por ejemplo, se valida que el correo tenga un formato válido y que al registrar un incidente se indique su tipo y nivel de emergencia.

Estas validaciones del Frontend no son suficientes por sí solas, ya que una persona podría enviar una petición directamente a la API. Por eso también realizamos las validaciones en el Backend antes de guardar o actualizar cualquier información. Si los datos no cumplen con lo esperado, se devuelve un mensaje de error y no se guarda la información en la base de datos.

Base de datos

Para almacenar la información utilizamos PostgreSQL. Se crearon las tablas necesarias para los diferentes módulos del sistema: usuario, incidente, historial_incidente, zona, refugio, recurso, asignacion_recurso, asignacion_personal y alerta.

Las tablas están relacionadas mediante llaves foráneas. Por ejemplo, un registro de historial_incidente está relacionado con un incidente mediante su id, mientras que una asignacion_personal se relaciona con un usuario y una zona.

La conexión entre el Backend y PostgreSQL se realizó utilizando la librería pg. También configuramos un pool de conexiones mediante variables de entorno para poder manejar las diferentes peticiones que recibe el sistema.

Manejo de errores

En el Backend utilizamos bloques try/catch para manejar los errores que pueden ocurrir durante las operaciones. De esta manera, si existe algún problema con la base de datos, con una petición o con la información solicitada, el servidor puede responder con el código correspondiente sin detenerse por completo. Entre las respuestas utilizadas están el 401 cuando no existe una autenticación válida, el 403 cuando el usuario no tiene permisos y el 500 para errores internos.

En el Frontend también se controlan estos errores. Por ejemplo, el interceptor detecta cuando se recibe un error 401 debido a que el token ya no es válido y permite cerrar la sesión. También se muestran mensajes al usuario cuando una operación no se puede completar.

Navegación

La navegación del sistema se realiza mediante las rutas de Angular. Estas están organizadas de acuerdo con las diferentes partes del proyecto: Incidentes, incluyendo Historial; Logística, donde están Recursos, Refugios y Asignaciones de recursos; Operaciones, con Alertas y Asignación de personal; y Core, donde se encuentran Usuarios, Zonas, Autenticación y Perfil.

También utilizamos guards para proteger las rutas. El authGuard verifica que exista una sesión antes de permitir el acceso a determinadas vistas, mientras que el adminGuard también comprueba que el usuario tenga el rol de Administrador. Por ejemplo, el módulo de Usuarios requiere permisos de Administrador. Si se intenta acceder a una ruta protegida sin cumplir las condiciones, el sistema redirige al usuario al login.

Formularios

Los formularios se utilizan para las acciones de creación y edición del sistema, como registrar un incidente, crear o editar un usuario, agregar un recurso o refugio, generar una alerta o realizar una asignación. Cada formulario tiene los campos necesarios según la información que se debe guardar y cuenta con validaciones para evitar enviar datos incompletos o incorrectos.

Consultas

El sistema permite consultar la información de diferentes maneras. Por ejemplo, se pueden listar los incidentes registrados, buscar usuarios por nombre, correo o rol desde el módulo de Gestión de Usuarios, consultar el historial de un incidente y revisar estadísticas generales desde el dashboard de logística.

Mensajes al usuario

También agregamos mensajes para que el usuario pueda saber qué está pasando en cada acción. Antes de eliminar información se muestra una confirmación para evitar borrados accidentales, cuando ocurre algún problema, se informa al usuario mediante un mensaje de error, esto también se aplica al inicio de sesión cuando las credenciales son incorrectas o cuando un usuario intenta realizar una acción para la que no tiene permisos.